CI/CD HANDBOOK

CI/CD Fundamentals

1. What is CI/CD ?

CI/CD is a set of practices that help developers integrate code frequently, automatically build, test and deliver software to users reliably and quickly.

Goal: Deliver value to users faster with high quality and less manual effort.

2. Continuous Integration (CI)

- Developers integrate code into a shared repository frequently.
- Every change is automatically built and tested.
- Helps detect bugs early.
- Improves code quality.

3. Continuous Delivery (CD)

- Code is always in a deployable state.
- After successful tests, code is ready to release to staging / pre-production.
- Release to production requires manual approval.

4. Continuous Deployment

- Every change that passes all tests is automatically deployed to production.
- No manual approval.
- Enables faster releases and better user experience.

5. CI vs CD vs Deployment

Aspect	Continuous Integration (CI)	Continuous Delivery (CD)	Continuous Deployment
Goal	Integrate & verify code	Prepare code for release	Release code to production automatically
Trigger	On every code commit	After CI (on successful tests)	After CD (auto on success)
Human Intervention	Not required	Approval required	Not required
Environment	Dev / Build Server	Staging / Pre-Prod	Production
Outcome	Verified & tested code	Release ready application	Live application for users
Frequency	Many times a day	Several times a day / week	Every change that passes

6. Benefits of CI/CD

Faster Delivery
Deliver features faster to users.

Better Quality
Early bug detection through automation.

Improved Collaboration
Team works together with shared processes.

Consistent Releases
Reliable and repeatable deployment process.

Higher Efficiency
Less manual work, more automation.

7. Real-world CI/CD Workflow

1. **Code Commit** — Developer writes & commits code
2. **Trigger Pipeline** — Push triggers CI/CD pipeline
3. **Build** — Dependencies installed, code built
4. **Test** — Automated tests (unit, integration, security)
5. **Package** — Artifact / Docker image created
6. **Deploy** — Deployed to target environment
7. **Monitor** — Monitor apps & get feedback

☆ Automate today, Deliver tomorrow ! ☆

VERSION CONTROL & GIT WORKFLOW

1. Git Basics

- Git is a **distributed** version control system.
- It helps **track changes** in code over time.
- Every developer works on a local copy of the repository.
- Git is **FAST**, **SAFE** and **FREE**.

Important Git Areas

- Working Directory: Files you are currently working on.
- Staging Area: Changes added and ready to commit.
- Repository (.git): All committed changes are stored here.

2. GitHub / GitLab / Bitbucket

- These are **remote platforms** to host git repositories.
- They provide collaboration, code review, issue tracking, CI/CD and many more features.

Common Commands:

`git clone <repo-url>`	→ Clone a repository
`git add .`	→ Add changes to staging area
`git commit -m "msg"`	→ Commit changes
`git push origin <branch>`	→ Push to remote repository
`git pull`	→ Pull latest changes

3. Branching Strategy

- Branching helps multiple developers work in parallel without affecting main code.
- Common Branches:
  - `main / master` → Stable production code
  - `develop` → Integration branch
  - `feature/*` → New features
  - `bugfix/*` → Bug fixes
  - `release/*` → Release preparation
  - `hotfix/*` → Critical fixes in production

4. Feature Branch

- Create a **new branch** for every new feature.
- Keeps the main branch **clean and stable.**
- Example:

```
git checkout -b feature/user-login
```
→ Create & switch to new branch

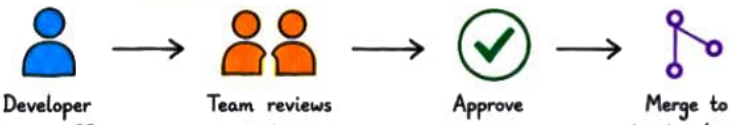
5. Git Flow (Simplified)

- feature/*: Work on new features.
- develop: Integrate & test features here.
- release/*: Prepare for production release.
- main: Production ready code.

6. Pull Requests (PR)

- After completing work, **create a PR.**
- Team **reviews** the code.
- Once approved, it is **merged.**
- Ensures better code quality.

PR Flow



7. Merge Conflicts

- Occurs when **same lines** of code are changed in different branches.
- Git cannot decide which change to keep.
- Developers must **resolve conflicts** manually.

Example

Branch A

```
int count = 10;
System.out.println("Hello");
```

Main Branch

```
int count = 5;
System.out.println("Hi");
```

Conflict

```
<<<<< HEAD (A)
int count = 10;
=====
int count = 5;
>>>>> main (B)
```

8. Code Review

- Review code **before merging.**
- Helps find **bugs** early.
- Improves code quality & team knowledge.

Good Code Review Checklist

- ✓ Functionality is correct
- ✓ Code is readable & clean
- ✓ Follows coding standards
- ✓ Proper error handling
- ✓ No security issues
- ✓ Tests are added / updated

⭐ Write Code. Review Code. Merge Code. Repeat! ⭐

ADVANCED PIPELINE & TOOLS

1. CI/CD Pipeline Stages (Detail)

1. **Plan** → Define requirements & plan the changes.
2. **Code** → Write code and commit to repository.
3. **Build** → Compile code and resolve dependencies.
4. **Test** → Run automated tests (unit, integration).
5. **Quality** → Code quality check (lint, static analysis).
6. **Package** → Package the application / create artifact.
7. **Deploy to Staging** → Deploy and test in staging environment.
8. **Deploy to Prod** → Deploy to production environment.
9. **Monitor** → Monitor app, logs, metrics & get feedback.

2. Types of Pipelines

- **Build Pipeline** : Compiles code and creates artifacts.
- **Delivery Pipeline** : Ensures code is always ready for release.
- **Deployment Pipeline** : Automatically releases to production.

Note:
CI happens on every commit.
CD ensures quick and reliable delivery / deployment. ☆

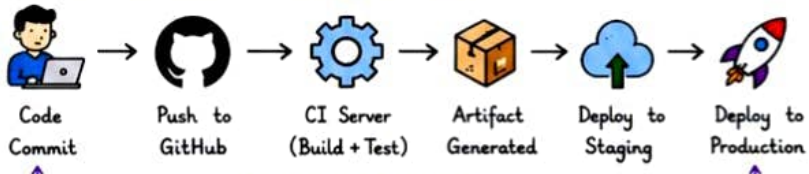
3. Popular CI/CD Tools

Jenkins	GitHub Actions	GitLab CI	CircleCI	Azure DevOps	Travis CI
Most popular open-source automation server.	CI/CD built into GitHub. Easy integration.	Built-in CI/CD, works great with GitLab.	Cloud-based CI/CD. Fast & easy configuration.	Microsoft's CI/CD tool with great ecosystem.	Easy CI/CD for open-source projects.

4. Artifacts & Environment

- **Artifact** : Output of the build (jar/war, exe, docker image etc.)
- **Environment** : Where the application runs.
  - **Dev** → For development
  - **Staging** → For testing/QA
  - **Prod** → For real users

5. Example CI/CD Pipeline Flow




```yaml
name: CI Pipeline          <-- Pipeline name
on: [push]                 <-- Trigger
jobs:                      <-- Job
  build:
    runs-on: ubuntu-latest
    steps:                 <-- Steps to run
      - uses: actions/checkout@v4
      - name: Build
        run: mvn clean package
      - name: Test
        run: mvn test
```

6. Pipeline as Code

- Pipelines are defined in code (YAML / JSON).
- Stored in repository.
- Version controlled, reviewable and reusable.
- Example : `.github/workflows/ci.yml`

7. Best Practices

- ☆ Keep commits small and frequent.
- ☆ Write automated tests.
- ☆ Keep pipeline fast and reliable.
- ☆ Use pipeline as code & version control.
- ☆ Monitor pipeline & fix failures quickly.

8. Key Takeaways

- ✓ CI improves code quality and team collaboration.
- ✓ CD ensures software is always deployable.
- ✓ Automation reduces manual work and errors.
- ✓ Faster delivery leads to happier users.
- ✓ Monitor, feedback and iterate continuously.

ADVANCED CONCEPTS of CI/CD

1. Triggers in CI/CD

- **Push** : When code is pushed to a branch.
- **Pull Request** : When PR is opened / updated.
- **Schedule (Cron)** : Run pipeline at specific time.
- **Manual** : Trigger pipeline manually if needed.
- **Tag** : When a new tag is created.

Example Triggers (GitHub Actions)

```yaml
on:
  push:
    branches: [ main, develop ]
  pull-request:
    branches: [ main ]
  schedule:
    - cron: "0 2 * * *"  # everyday at 2 AM
  workflow_dispatch:     # manual trigger
```

2. Environments in CD

- **Dev** → For developers (internal testing)
- **Staging** → QA / UAT testing
- **Prod** → Live users

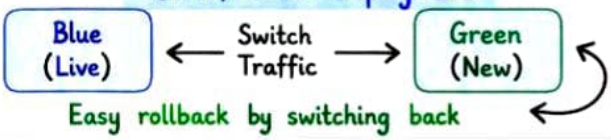
★ Always deploy through environments in order:

Dev → Staging → Prod

3. Strategies for Deployment

- **Recreate** : Stop old version, start new version.
- **Rolling Update** : Update instances one by one.
- **Blue/Green** : Two identical environments.
- **Canary** : Release to small % of users first.
- **Feature Flag** : Hide / show features without deploy.

Blue / Green Deployment



Easy rollback by switching back

4. Quality Gates in Pipeline

Code	Build	Test	Quality Gate	Package	Deploy
• Lint	• Compile	• Integration Tests	• SonarQube	• Build Artifact	• Deploy to Env
• Format	• Unit Tests	• Coverage Check	• Code Quality	• Versioning	• Smoke Tests
• SAST Scan	• Dependency Check	• E2E Tests	• Maintainability	• Store Artifact	• Health Check

★ If any Quality Gate fails → Pipeline stops. Fix issues and re-run.

5. Secrets & Variables

- Never hard-code secrets in code.
- Use Secrets / Variables provided by CI tool.
- Types:
  - **Secret** → Encrypted (passwords, keys, tokens)
  - **Variable** → Non-sensitive values (url, env, config)

Example (GitHub Actions)

```yaml
env:
  DB_URL: ${{ secrets.DB_URL }}
  API_KEY: ${{ secrets.API_KEY }}
  ENV: ${{ vars.ENVIRONMENT }}
```

6. Artifact Management

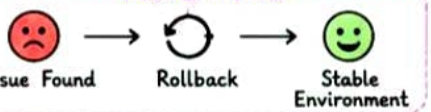
- Artifacts are the output of build (jar, exe, docker image).
- Store artifacts in a repository (JFrog, Nexus, GitHub Packages).
- Use versioning for traceability.

Tool	Use
JFrog Artifactory	Store & manage all types of artifacts
Nexus Repository	Maven, npm, docker, etc.
GitHub Packages	Store docker images, packages
AWS S3 / GCS	Store build files, reports, backups

7. Rollback Strategy

- Always have a rollback plan.

Rollback Flow



Tips

- ✓ Monitor Pipeline regularly
- ✓ Keep Build Small & Fast
- ✓ Automate as much as possible

TESTING IN CI/CD & QUALITY

1. Types of Testing in Pipeline

- Unit Testing → Test individual units / functions.
- Integration Testing → Test interactions between modules.
- API Testing → Test APIs (endpoints, status, response).
- UI / E2E Testing → Test the full user flow.
- Performance Testing → Load, stress, scalability.
- Security Testing → Vulnerability scan, dependency scan.

Where they run?

- ✓ Unit → Early in pipeline (fast)
- ✓ Integration → After build
- ✓ E2E / UI → Before production
- ✓ Performance / Security → Before release

☆ The earlier you test, the cheaper it is to fix!

2. Quality Checks in CI

Lint	Static Analysis	Unit Tests	Coverage	Dependency Check	Build Verification
Check code style & formatting.	Find bugs, code smells, vulnerabilities.	Verify logic of individual units.	Ensure sufficient test coverage.	Check vulnerable dependencies.	Ensure build is successful.

3. Test Automation Best Practices

- ✓ Automate repeatable tests.
- ✓ Keep tests independent and reliable.
- ✓ Follow AAA pattern (Arrange, Act, Assert).
- ✓ Use meaningful test data.
- ✓ Run tests in parallel to save time.
- ✓ Maintain tests like production code.

AAA Pattern

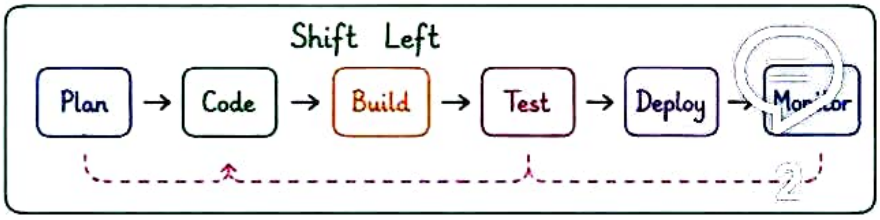
- Arrange → Setup data & preconditions
- Act → Perform the action
- Assert → Validate the result

4. Quality Tools You Can Use

Category	Tools	Use
Linting	ESLint, Pylint, Checkstyle	Check code style & catch common issues
Static Analysis	SonarQube, CodeQL, PMD	Find bugs, code smells, security issues
Unit Testing	JUnit, TestNG, PyTest, NUnit	Run unit tests automatically
Coverage	JaCoCo, Istanbul, Coverage.py	Measure test coverage
Security	Snyk, OWASP ZAP, Trivy	Scan vulnerabilities in code & dependencies
API Testing	Postman, Newman, RestAssured	Test APIs automatically
UI / E2E	Cypress, Playwright, Selenium	Test real user workflows

5. Shift Left Testing

- Test early in the SDLC (left side).
- Find bugs early → Save time & cost.
- Improves quality and confidence.



Popular DevOps Tools

1. CI/CD Tools Overview

- **GitHub Actions** — CI/CD built into GitHub. Easy setup with YAML.
- **Jenkins** — Open-source automation server. Highly customizable.
- **GitLab CI/CD** — Built-in CI/CD with GitLab. Great for DevSecOps.
- **Azure DevOps** — End-to-end DevOps platform by Microsoft.
- **CircleCI** — Fast and simple CI/CD. Great performance.
- **Travis CI** — Easy CI for GitHub projects. Simple YAML config.
- **Docker** — Containerization platform. Package app with dependencies.
- **Kubernetes** — Container orchestration. Manage & scale containers.
- **Argo CD** — GitOps continuous delivery tool for Kubernetes.
- **SonarQube** — Code quality & security analysis tool.

2. Tool Comparison

Tool	Type	Ease of Use	Speed	Best For	Cost
GitHub Actions	CI/CD	★★★★★	★★★★☆	GitHub Projects	Free for public Paid for private
Jenkins	Automation Server	★★☆☆☆	★★★☆☆	Complex Pipelines	Free
GitLab CI/CD	CI/CD	★★★★☆	★★★★☆	GitLab Users	Free (CE) Paid (EE)
Azure DevOps	DevOps Platform	★★★★☆	★★★★☆	Microsoft Ecosystem	Free + Paid Plans
CircleCI	CI/CD	★★★★☆	★★★★☆	Fast Builds	Free + Paid Plans
Travis CI	CI	★★★☆☆	★★★☆☆	Small Projects	Free for OSS
Docker	Container Platform	★★★★☆	★★★★☆	Containerization	Free + Paid Plans
Kubernetes	Orchestration	★★★☆☆	★★★★★	Scaling Apps	Free
Argo CD	CD (GitOps)	★★★☆☆	★★★☆☆	K8s Deployments	Free
SonarQube	Quality Analysis	★★★★☆	★★★☆☆	Code Quality	Free + Paid Plans

3. Complete CI/CD Toolchain Flow

4. Best Tool Selection Guide

Using GitHub?	→ Use GitHub Actions
Using GitLab?	→ Use GitLab CI/CD
Need highly customizable?	→ Use Jenkins
Working in Microsoft stack?	→ Use Azure DevOps
Need fast & simple CI?	→ Use CircleCI
Small GitHub projects?	→ Use Travis CI
Using containers?	→ Use Docker
Need to orchestrate?	→ Use Kubernetes
Want GitOps CD?	→ Use Argo CD
Need code quality?	→ Use SonarQube

5. Production Tips

- ✓ Keep your pipelines **fast** → Parallel jobs, cache dependencies.
- ✓ **Fail fast** → Stop pipeline on first critical failure.
- ✓ Use **secrets management** → Never hardcode credentials.
- ✓ **Scan early** → Security & quality checks in every PR.
- ✓ Use **environments** → Dev, Staging, Prod with approvals.
- ✓ **Monitor everything** → Logs, metrics, alerts.
- ✓ **Automate rollbacks** → Save time & reduce downtime.

> ★ Good pipelines are not just about deployment, it's about reliability, security & speed.

6. CI/CD Best Practices

- Small **commits** & **frequent** pushes.
- **Automate** builds, tests & deployments.
- Keep configurations in **version control**.
- Use **IaC** for environments.
- **Document** pipelines & keep it simple.

Golden Rule

Automate everything

7. Remember

- CI → Build & test your code.
- CD → Deliver your code.

Security, Monitoring & Advanced Concepts

1. Security in CI/CD

- Never store **secrets** in code or repo.
- Use **Secrets Manager** (Vault, AWS Secrets Manager, GitHub Secrets).
- Use **least privilege** access for service accounts.
- Scan **dependencies** for vulnerabilities.
- Sign **artifacts** & verify integrity.
- Use **SAST**, **DAST**, and **Dependency Scanning**.
- Enable **branch protection** & code reviews.
- **Audit logs** & monitor pipeline activities.

Secrets Flow

Secrets are never stored in code or logs.

2. Monitoring & Observability

- Monitor **pipeline runs**, duration, success/failure.
- Track **deployments**, rollbacks & changes.
- Collect **logs, metrics & traces**.
- Set up **alerts** for failures & performance issues.
- Use **dashboards** for visibility.
- **Tools:** Prometheus, Grafana, ELK Stack, Datadog.

What to Monitor?

- ✓ Pipeline Success Rate
- ✓ Build Duration
- ✓ Test Results
- ✓ Deployment Status
- ✓ Error Rate
- ✓ Response Time
- ✓ Resource Usage
- ✓ Rollbacks

3. Advanced Concepts

- ☆ **Pipeline as Code** → Define CI/CD using YAML.
- ☆ **Matrix Builds** → Run tests on multiple env/versions.
- ☆ **Parallelism** → Run jobs in parallel to save time.
- ☆ **Caching** → Cache dependencies & artifacts.
- ☆ **Conditional Execution** → Run jobs based on conditions.
- ☆ **Reusable Workflows** → Create & reuse workflow templates.
- ☆ **Canary Deployment** → Release to small % users first.

Example: Matrix Build

OS \ Version	Node 18	Node 20	Node 22
Ubuntu	✓	✓	✓
Windows	✓	✓	✓
macOS	✓	✓	✓

Run tests on multiple OS & Node versions in parallel.

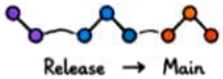
4. Common Pipeline Patterns

Basic Pipeline

Build → Test → Deploy

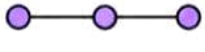
GitFlow Pipeline

Feature → Develop → Release → Main



Trunk Based Pipeline

Commit → Build → Deploy

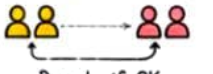


Blue/Green Deployment

Live (Blue) New (Green)

Canary Deployment

v1 (Stable) v2 (Canary)



5. CI/CD Anti-Patterns

- ✗ Long-running pipelines.
- ✗ Flaky tests.
- ✗ Manual approvals everywhere.
- ✗ No rollback strategy.
- ✗ Ignoring security scans.
- ✗ Not monitoring pipelines.

Avoid them! ☹

They slow you down and risk your production.

6. Checklist for a Great Pipeline

- ✓ Fast & Reliable
- ✓ Automated & Repeatable
- ✓ Secure by Default
- ✓ Well Tested
- ✓ Proper Notifications
- ✓ Monitored & Alerted
- ✓ Easy Rollback
- ✓ Well Documented
- ✓ Cost Efficient
- ✓ Developer Friendly

7. Key Takeaways

- ☑ Security first, always.
- ☑ Monitor everything.
- ☑ Automate smartly.

Infrastructure, Environments & Scaling

1. Environments in CI/CD

- Dev → Developers build & test.
- QA / Staging → QA validates and testing.
- UAT → Business users validate.
- Prod → Live users, real traffic.

Environment Promotion Flow

Dev → QA / Staging → UAT → Prod

(Automated where possible)

2. Infrastructure as Code (IaC)

- Manage infrastructure using code.
- Consistent, repeatable & version controlled.
- Popular Tools :

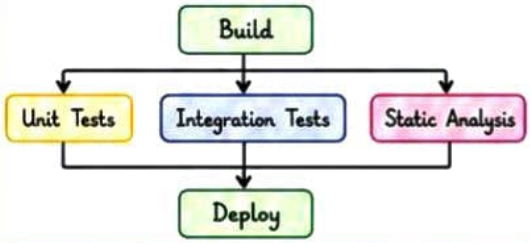
Terraform · Azure Bicep · AWS CloudFormation · Pulumi · Ansible

Treat your infrastructure like code.
Version it, review it, test it !

3. Scaling Your Pipelines

- Use parallel jobs to reduce build time.
- Split tests into multiple stages.
- Use caching for dependencies & layers.
- Run only impacted tests (smart build).
- Use self-hosted runners when needed.

Parallel Execution Example

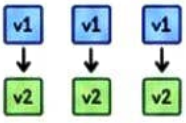


4. Deployment Strategies

Recreate
Stop old → Start new

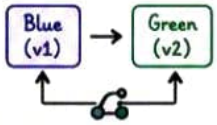
+ Simple
− Downtime

Rolling Update
Replace instances



+ No downtime
− Slow process

Blue/Green
Switch traffic

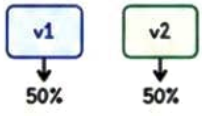


+ Zero downtime
− Double infra cost

Canary
Release to small %

+ Safe release
− Complex setup

A/B Testing
Compare versions



+ Data driven
− Needs analytics

5. Artifact Storage Options

Tool	Use
JFrog Artifactory	Store binaries, libs, docker images
Nexus Repository	Maven, npm, docker & more
AWS S3	Store build files, backups, artifacts
GitHub Packages	Docker, npm, maven packages
Azure Artifacts	Universal packages, feeds

6. Notifications & Alerts

- Send notifications for build status.
- Alert on failures & performance issues.
- Tools :

Slack · Teams · Email · PagerDuty

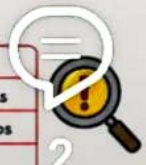
Don't just build it, know what's happening !

7. Cost Optimization Tips

- ✓ Use self-hosted runners for heavy workloads.
- ✓ Cache dependencies & docker layers.
- ✓ Run jobs only on changes (paths/filters).
- ✓ Use spot instances & auto-scaling.

8. Common Issues & Solutions

Issue	Solution
Flaky Tests	Stabilize tests, retry with limits
Slow Builds	Parallelize, cache, optimize steps
Failed Deploys	Rollback & improve checks
High Costs	Optimize runners, storage, scaling



Advanced Performance Practices & Reliability

1. Advanced CI/CD Practices

- ★ **Feature Flags** → Safely release features.
- ★ **Immutable Artifacts** → Never change, only promote.
- ★ **Ephemeral Environments** → Create temp env for PRs.
- ★ **GitOps** → Manage infra & apps via Git.
- ★ **Policy as Code** → Enforce security & compliance.
- ★ **Shift Left** → Test early, find bugs early.

Feature Flag Flow

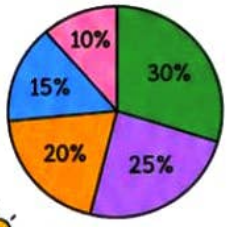
Code Deployed → Feature Flag OFF (Hidden) → Enable Flag for Users → Monitor & Iterate

★ Release to users gradually & safely.

2. Performance Optimization

- ✓ Run tests in **parallel**.
- ✓ **Cache** dependencies & build layers.
- ✓ Use minimal **Docker images** (Alpine).
- ✓ **Reuse** build agents & workspaces.
- ✓ **Optimize** pipeline steps & reduce IO.
- ✓ Use **self-hosted runners** for heavy jobs.

Example: Pipeline Time Breakdown.



Optimize the biggest slices first!

3. Reliability Best Practices

- ✓ Make pipelines **idempotent** (safe to re-run).
- ✓ Use **retry logic** for flaky steps.
- ✓ Add **timeouts** to prevent hanging jobs.
- ✓ Keep pipelines small, focused & **modular**.
- ✓ Use **health checks** after deployment.
- ✓ Automate **rollbacks** on failure.

Idempotent Example

Run Job → Same result every time.

4. Handling Failures

Failure Type	What to Do
Build Failure	Fix code, push & re-run.
Test Failure	Analyze logs, fix tests/code.
Timeout / Hang	Add timeouts, optimize steps.
Infra Failure	Retry / Failover / Alert team.
Deployment Failure	Auto rollback & notify.

★ Fail fast, alert, fast, fix fast!

5. Security in CI/CD

- Scan code for **secrets** (GitLeaks, TruffleHog).
- Use **dependency scanning** (Snyk, OWASP).
- Scan **container images** for vulnerabilities.
- Enforce **least privilege** access.
- Sign **artifacts** (Provenance / SBOM).
- **Audit** pipeline changes & access.

Security Scanning Flow

Code / Image → Scan → Report → Fail if Critical

6. Observability in Pipelines

- ✓ Log everything.
- ✓ Use structured logs.
- ✓ Add metrics for pipeline health.
- ✓ Track **DORA Metrics**.

★ You can't improve what you don't measure.

DORA Metrics

- Deployment Frequency
- Lead Time for Changes
- Change Failure Rate
- MTTR (Mean Time to Recover)

7. Multi-Environment Strategy

Dev	QA	Staging	Prod
• Fast	• Validate	• Mirror Prod	• Stable
• Frequent	• Test	• Final Test	• Secure
• Developers	• Quality	• Approvals	• Users

→ Promote the same artifact across environments.

8. Release Strategies

Rolling Update	Blue/Green	Canary	A/B Testing
Update pods one by one.	Blue → Green. Switch traffic to new version.	5% ↔ 100%. Release to few users then scale.	A 50% / B 50%. Compare version & choose best.

9. Pro Tips

- Automate everything you can.
- Keep pipelines simple & repeatable.
- Continuously improve & refactor.

10. Remember

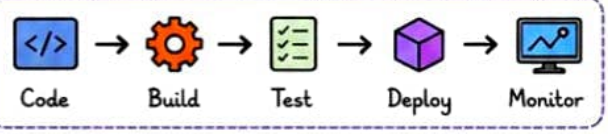
- Fast pipelines = Happy teams.
- Reliable pipelines = Happy users.
- Quality & Security = Long term success.

CI/CD HANDBOOK

FINAL SUMMARY & ROADMAP

1. CI/CD in a Nutshell

CI/CD is the practice of automating the integration, testing, delivery & deployment of code.



CI (Continuous Integration)
- Developers integrate code
- Frequent builds
- Automated tests
- Early bug detection

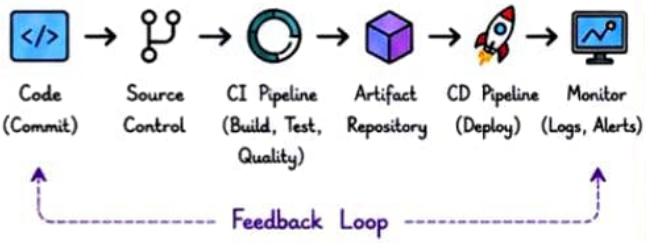
CD (Continuous Delivery/Deployment)
- Deliver code to staging
- Get manual/auto approval
- Deploy to production
- Faster releases

2. CI/CD Benefits

- ✓ Faster Delivery → Ship features quickly
- ✓ Better Quality → Automated testing & checks
- ✓ Early Feedback → Catch issues early
- ✓ Lower Risk → Small changes, easy rollback
- ✓ Team Productivity → Less manual work
- ✓ Visibility → Know what's happening
- ✓ Reliability → Consistent & stable releases

Automate the boring stuff, so your team can build awesome things!

3. End-to-End CI/CD Pipeline (Recap)



4. Key Metrics to Track

Metric	Why it Matters
DORA Metrics	Measure pipeline performance
Deployment Frequency	How often you release
Lead Time for Changes	From commit to production
Change Failure Rate	% of changes causing failure
MTTR (Recovery Time)	How fast you recover

5. CI/CD Maturity Model

1	2	3	4	5
Manual	Automated	Repeatable	Measured	Optimized
Manual build & deployment	Automated build & test	Standardized pipelines	Metrics driven improvements	Continuous improvement at scale

Always aim for small, consistent improvements.

6. Common Pitfalls to Avoid

- ✗ Over-complicated pipelines.
- ✗ Skipping tests for speed.
- ✗ Ignoring security & secrets.
- ✗ Big, risky deployments.
- ✗ No monitoring after deployment.
- ✗ No rollback or recovery plan.

Keep it Simple, Secure & Observable!

7. CI/CD Roadmap for You

1. Learn Basics → Git, Linux, Bash, CI/CD Concepts
2. Version Control → Git, GitHub/GitLab
3. Build & Test → Automate Build, Unit Tests
4. CI Pipelines → GitHub Actions / GitLab CI
5. Artifacts → Store & Manage Artifacts
5. Deploy → Staging → Production
6. Infrastructure → Docker, Kubernetes, IaC
7. Monitoring → Logs, Metrics, Alerts
8. Security → Secrets, Scans, Compliance
10. Improve → Optimize, Automate, Iterate

Keep learning. Keep building. Keep shipping!

8. CI/CD Cheat Sheet

Run all tests	→ npm test / mvn test
Build project	→ npm run build / mvn package
Build Docker image	→ docker build -t myapp .
Run container	→ docker run -d -p 8080:8080 myapp
Push image	→ docker push myrepo/myapp
Deploy to K8s	→ kubectl apply -f k8s/
View logs	→ kubectl logs -f <pod-name>
Rollback	→ kubectl rollout undo deployment/<name>

9. Golden Rules

- Automate everything that can be automated.
- Test early. Test often. Test everything.
- Small changes. Frequent releases.
- Observe, measure & improve continuously.

10. Final Note

CI/CD is not just about tools, it's a culture of Collaboration, Automation, Quality & Continuous Improvement. Make it a habit. Make it awesome!